

Programmeringsspråket Gofer

Kjell Post

13 september 2026

1 Introduktion

- **Gofer** är ett *funktionellt* språk. Funktioner kan skickas som argument, returneras som resultat och lagras i variabler. Den huvudsakliga kontrollstrukturen är rekursion.
- **Gofer** är *interaktivt*. Uttryck analyseras och körs varefter värdet skrivs ut.
- **Gofer** är *starkt typat*. Varje giltigt uttryck har en typ som härleds av systemet. Stark typning garanterar att typfel inte uppstår under körning.
- **Gofer** har *lat evaluering* vilket innebär att argument till funktioner inte evalueras förrän de behövs.

Gofer påminner mycket om Haskell, ett annat funktionellt språk med lat evaluering. Till skillnad från Haskell kan **Gofer** köras på en mängd datorer, t ex hemma på din PC eller Mac. **Gofer** är gratis och implementationer för olika maskiner kan hämtas med ftp från <ftp.cs.chalmers.se> i katalogen `pub/haskell/gofer`.

Denna rapport är avsedd som en informell introduktion till **Gofer** och är absolut ingen fullständig definition av språket. För mer information, se [1] som finns på ovan nämnda ftp-arkiv.

2 Interaktion med Gofer

Gofer startas i UNIX med kommandot

```
$ gofer
```

Om detta inte fungerar, kontakta systemansvarig.

När systemet startat får du en prompt där du kan mata in uttryck:

```
? 3+2
5
(3 reductions, 7 cells)
```

Systemet evaluerar uttrycket och visar svaret, tillsammans med lite information om hur mycket resurser som krävdes.

Andra kommandon, för att t ex ladda in filer och avsluta **Gofer**, inleds med ":". En sammanställning av dessa kommandon fås genom att skriva ":"? vid prompten. Vi kommer att ta upp några av dessa senare.

Olika fel kan uppstå under evalueringen. De vanligaste felen är syntaxfel, typfel och runtime-fel. Du är säkert bekant med syntax- och runtime-fel från andra språk, men ett typfel är kanske mer obekant. Vi kommer att diskutera typer längre fram, så vi nöjer oss med att konstatera att ett typfel uppstår när ett värde används på fel sätt, t ex genom att addera 3 och True:

```
? 3+True
```

```
ERROR: Type error in application
*** expression      : 3 + True
*** term            : 3
*** type            : Int
*** does not match  : Bool
```

3 Uttryck, värden och typer

Låt oss börja med en översikt av de grundläggande typerna. I **Gofer** är en typ en mängd av värden. Till exempel så är heltalen en typ, liksom tecken och booleska värden. Givet två typer σ och τ så är mängden av ordnade par, med den vänstra komponenten av typen σ och den högra av typen τ , också en typ. En annan viktig typ är mängden av funktioner som mappar en typ till en annan. Utöver detta tillåter **Gofer** definition av egen-definierade typer, något som vi kommer att studera senare. Vi kommer att illustrera typsystemet med olika exempel.

3.1 Booleska

Typen Bool består av typerna False och True. Funktionerna not, && och || implementerar logisk negation, konjunktion och disjunktion, respektive.

```
? not False
True
? True && False
False
```

Också värt att nämna är det villkorliga uttrycket if b then e_1 else e_2 , där b måste vara av typen Bool. Dessutom måste typen på e_1 och e_2 vara lika, ty denna typ definierar resultat av hela if-uttrycket.

```
? if True then 1 else False
```

```
ERROR: Type error in conditional
*** expression      : if True then 1 else False
```

```
*** term          : False
*** type          : Bool
*** does not match : Int
```

3.2 Heltal

Typen `Int` omfattar alla positiva och negativa heltal.

```
? if 4*5 == 60/3 then 17 else -42
17
```

De vanliga aritmetiska operationerna `+`, `-`, `*`, `/` samt unärt `-` finns naturligtvis.

Referenser

[1] Mark Jones. *An Introduction to Gofer*, February 1995.